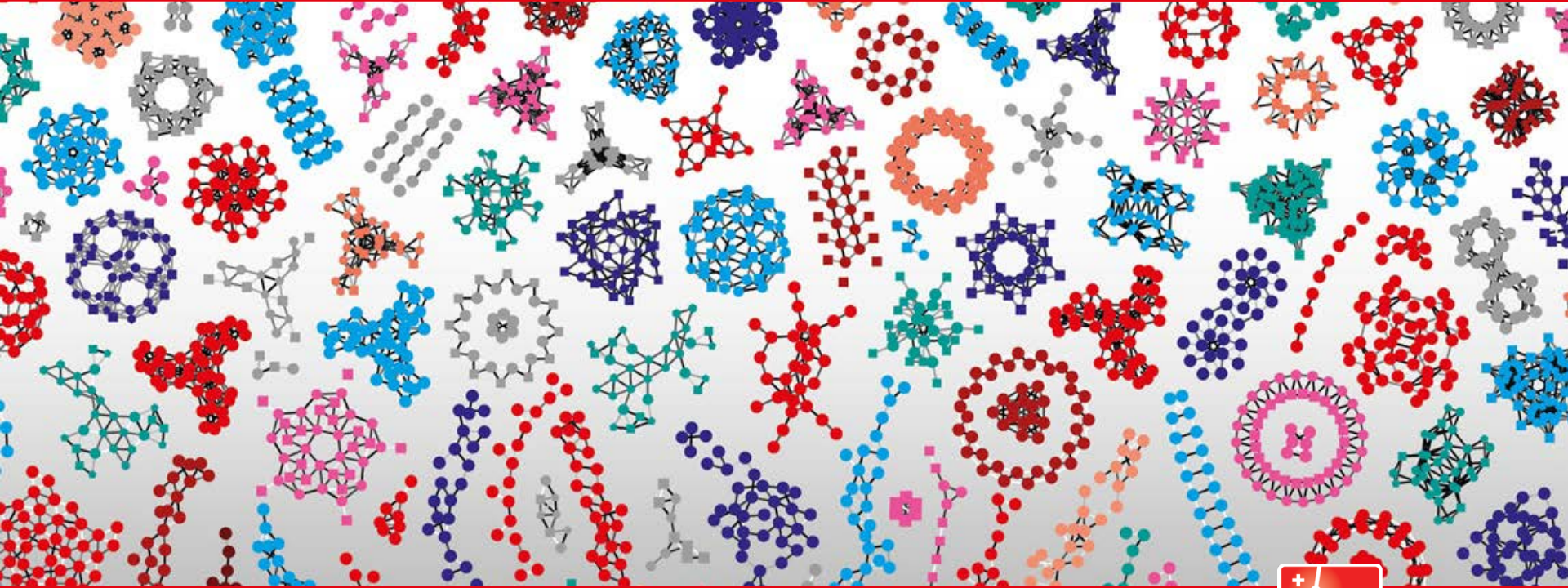




Swiss Institute of
Bioinformatics

Advanced R Namespaces, profiling, ...

Frédéric Schütz (Frederic.Schutz@sib.swiss)

Namespaces

**What happens when
several packages
define the same function?**

Example: the Hmisc package

Attaching package: 'Hmisc'

The following objects are masked from 'package:base':

format.pval, round.POSIXt, trunc.POSIXt, units

Each R package has its own **namespace** for objects, so that several packages can provide the same function without any interference.

Hmisc

`format.pval`

`units`

`...`

stats

`var`

`median`

`...`

base

`summary`

`format.pval`

`is.factor` `...`

`mean`

When looking for a function, R follows a **search path** through the namespaces until it finds the first occurrence of the function it is looking for:

```
> search()  
[1] ".GlobalEnv"      "package:Hmisc"    "package:ggplot2"  
[4] "package:Formula" "package:survival"  "package:lattice"  
[7] "package:stats"    "package:graphics" "package:grDevices"  
[10] "package:utils"    "package:datasets" "package:methods"  
[13] "Autoloads"        "package:base"
```

Functions from different packages can be differentiated using `::`

```
> Hmisc::format.pval(0.05)
[1] "0.05"
> base::format.pval(0.05)
[1] "0.05"
```

If you display the code of the function from a package, the namespace is printed afterwards

```
> Hmisc::format.pval
function (x, pv = x, digits = max(1, .Options$digits - 2),
        eps = .Machine$double.eps, na.form = "NA", ...)
{
  if ((has.na <- any(ina <- is.na(pv))))
    ...
  r
}
<bytecode: 0x55be43c290d0>
<environment: namespace:Hmisc>
```


If you display the code of the function from a package, the namespace is printed afterwards

```
> base::format.pval
function (x, pv = x, digits = max(1, .Options$digits - 2),
        eps = .Machine$double.eps, na.form = "NA", ...)
{
  if ((has.na <- any(ina <- is.na(pv))))
    ...
  r
}
<bytecode: 0x55be43c290d0>
<environment: namespace:base>
```

This allows the redefinition of a function, while still allowing access to its original version:

```
# My own summary
summary.default <- function( data ) {
  # Start by getting the original summary
  originalsummary <- base::summary.default(data)

  # Then we modify the output as we want
  ...
}
```

After deleting the new function, the original one remains available.

Be careful to avoid recursion

Make sure the function does not call itself !

```
# My own summary
summary.default <- function( data ) {
  # Start by getting the original summary
  originalsummary <-      summary.default(data)

  # Then we modify the output as we want
  ...
}
> summary(1:10)
Error: C stack usage 7970068 is too close to the limit
```

Example

}

,

,

,

,

,

,

,

Example: redefining the addition

```
[1] 2.1  
> rm(`+`) # Go back to a sane version of the addition.  
> 1+1  
[1] 2
```

A package can choose to make a function available outside its namespace by exporting it.

Otherwise, by default, the code is only available to other functions from this package.

Example: the `t.test` function in package `stats`

```
> t.test
```

```
function (x, y, data.name, mu = 0, alternative = "two.sided",
```

Example: the `t.test` function in package `stats`

```
> t.test
```

```
function (x, y, data = NULL, ...)
```

```
[1] t.test.default t.test.formula
```

```
see '?methods' for accessing help and source code
```


Example: the `t.test` function in package `stats`

```
> t.test
function (x, ...)
UseMethod("t.test")
<bytecode: 0x55ccd563e0c0>
<environment: namespace:stats>

> methods(t.test)
[1] t.test.default* t.test.formula*
see '?methods' for accessing help and source code
> t.test.default
Error: object 't.test.default' not found
```

The package exports `t.test` (available from outside) but not `t.test.default`, which you are supposed to call through the generic function `t.test` only.

How to access a non-exported function ?

To get the source code:

```
> getAnywhere(t.test.default)
```

A single object matching 't.test.default' was found

It was found in the following places

registered S3 method for t.test from namespace stats

namespace:stats

with value

```
function (x, y = NULL, alternative = c("two.sided", "less",  
"greater"),  
  mu = 0, paired = FALSE, var.equal = FALSE, conf.level = 0.95,  
  ...)  
{  
  alternative <- match.arg(alternative)  
  if (!missing(mu) && (length(mu) != 1 || is.na(mu)))
```

How to access a non-exported function ?

To run it:

```
> stats::t.test.default()
```

```
Error: 't.test.default' is not an exported object from  
'namespace:stats'
```

How to access a non-exported function ?

To run it:

```
> stats::t.test.default()  
Error: 't.test.default' is not an exported object from  
'namespace:stats'
```

```
> stats:::t.test.default()  
Error in stats:::t.test.default() :  
  argument "x" is missing, with no default
```

However, there is usually a good reason for the function not to be exported

getAnywhere: finds all namespaces containing a given function

```
package:base
```

```
registered S3 method for format from namespace Hmisc
```

```
namespace:base
```

```
namespace:Hmisc
```

Use [] to view one of them

```
> getAnywhere(format.pval)[1]
```

```
function (pv, digits = max(1L, getOption("digits") - 2L),
```

```
...
```

```
<environment: namespace:Hmisc>
```

Remember...

How does R store both the `c()` function and the `c` vector, and how does it differentiate between them ?

```
> c=c(c=c)
> c=c(c="c")
> c
  c
" c"
```

The vector and the function belong to different namespaces

```
> c=c(c=c)
> c=c(c="c")
> getAnywhere(c)
2 differing objects matching 'c' were found
in the following places
  .GlobalEnv
  package:base
  namespace:base
Use [] to view one of them
```

**If you try to run a function,
R will ignore all other types
of variables it will find**

Environments

An environment is a data structure that contains R objects.

Each package or function has its own environment, which defines the variables it has access to.

Each environment also has a parent environment; variables in the parent environment are also available to the function.

```
> environment()  
<environment: R_GlobalEnv>  
> f <- function() { environment() }  
> f()  
<environment: 0x55b02effa7e8>
```

Lexical scoping

```
> a
[1] 2
> f <- function() { print(a) }
> f()
[1] 2
> f <- function() { a <- 1; print(a) }
> a
[1] 2
> f()
[1] 1
> a
[1] 2
```

< .

VS

=

R provides 5 assignment operators:

```
?assignOps
```

```
Description
```

```
Assign a value to a name.
```

```
Usage
```

```
x <- value
```

```
x <<- value
```

```
value -> x
```

```
value ->> x
```

```
x = value
```



Assignment operators: <- vs =

- Originally, R would only accept <- for assignment
- This choice has a historical origin in the APL programming language, at a time where "←" was an actual key on the keyboard
- The "=" operator was added in 2001, for improving compatibility with other languages.
- Both Hadley Wickham's and Google's styleguides suggest using "<-" only, and so does the R community in general
- The two operators are mostly interchangeable
- There are a few exceptions, though...



Assignment operators: <- vs =

- Function parameters can only be specified with an "=":

```
mean(data, na.rm=TRUE)    # work
```

```
mean(data, na.rm<-TRUE)   # does not work
```

- However, if you want to specify an assignment within a parameter, you must use <-
- For example, if you want to compute an expression, store it and measure its execution time simultaneously:

```
system.time(result<-expression) # works
```




Assignment operators: <- vs =

- Using `result=expression` would not work, as the `system.time()` function does not accept a `result` parameter
- An alternative way of doing this would be:
`system.time( (result=expression) )`
- More generally, <- can be used everywhere, while = can only be used at the "top level"
- For example:

```
if (x <- 0) 1 else 0    # works
if (x = 0)  1 else 0    # does not work
```
- One reason for this: confusing `x=0` and `x==0` is one of the most common mistake in other programming languages
- But in most cases, you can probably avoid using such a construct anyway...

Local vs global variables



Local vs global variables



Local vs global variables

```
> m <- 1  
> f <- function() { m <<- m + 1 }  
> f()  
> m  
[1] 2
```

The "<<-" operator forces the assignment to work on the global m variable, and not on a local variable that exists only inside the function.

the attach() command

**Do you use
the attach() command ?**

The attach command adds a dataframe or list into the search path

```
✓ ualapa
```

```
[1] 1
```

The rule is simple

The rule is simple:

Never use `attach()`

Avoid the attach command

```
> attach(data)                # a = ?  
# Warning message displayed  
> rm(a)                       # a = ?  
> detach(data)                # a = ?  
> detach(data)                # a = ?  
> attach(data)                # a = ?  
> rm(list = ls())             # a = ?  
> detach(data)                # a = ?
```

Avoid the attach command

```
> attach(data)                # a = 4
# Warning message displayed
> rm(a)                        # a = 4 (error message)
> detach(data)                 # a = 1
> detach(data)                 # Error message
> attach(data)                 # a = 4
> rm(list = ls())              # a = 4
> detach(data)                 # Error message
```

What happens with the search path

```
< attach(data)
> search()
[1] ".GlobalEnv"      "data"             "package:stats"
[4] "package:graphics" "package:grDevices" "package:utils"
[7] "package:datasets" "package:methods"   "Autoloads"
[10] "package:base"
```

What happens with the search path

```
> search()
[1] ".GlobalEnv"      "data"             "data"
[4] "package:stats"   "package:graphics" "package:grDevices"
[7] "package:utils"   "package:datasets" "package:methods"
[10] "Autoloads"       "package:base"
```

Attached data remains even after deleting everything

```
> search()
[1] ".GlobalEnv"      "data"             "data"
[4] "package:stats"    "package:graphics" "package:grDevices"
[7] "package:utils"    "package:datasets" "package:methods"
[10] "Autoloads"        "package:base"
```

Use «with», «within» or «transform» instead

```
> with(clinicaldata, plot( genotype, phenotype ) )
```

```
# Equivalent to
```

```
> plot(clinicaldata$genotype, clinicaldata$phenotype)
```

Use «with», «within» or «transform» instead

```
> new <- within(clinicaldata, genotype <- log2(genotype))  
> new  
  phenotype    genotype  
1 0.8142518 -0.09733194  
2 0.9287772 -1.53048032  
3 0.1474810 -0.90762854
```


Use «with», «within» or «transform» instead

```
> head(clinicaldata, 3)
  phenotype  genotype
1 0.8142518 0.9347601
2 0.9287772 0.3461621
3 0.1474810 0.5330606

> transform(clinicaldata, genotype = log2(genotype))

# Equivalent to
> clinicaldata$genotype <- log2(clinicaldata$genotype)
```

Using `transform()` is clearer than using the direct command, but less flexible than using `within()`.

One more question...

**What does the `library()`
command do ?**

Is there any practical difference between these two loops ?

```
set.seed(1)
n <- 5000; m <- 5000
```

```
}  
}
```

```
# Loop 2  
for (i in 1:ncol(a)) {  
  for (j in 1:nrow(a)) {  
    b <- a[j,i]  
  }  
}
```

Efficient programming in R

Techniques used in other languages are often inefficient in R.

In particular, they tend not to scale when the size of data increases.

R itself is not the fastest possible language.

Finding which method is efficient or not is far from obvious (in R or any programming language).

Measuring the time used by an expression (I)

Use the commands:

```
library(microbenchmark)  
microbenchmark(expression1, expression2, ...)
```

which runs the expressions 100 times (by default) and returns a summary of the running time.

Which one is fastest ?





Which one is fastest ?

```
> microbenchmark( sqrt(x), x^0.5 )
```

Unit: microseconds

expr	min	lq	mean	median	uq	max	neval	cld
sqrt(x)	1.314	1.3720	1.80951	1.4190	1.460	33.621	100	a
x^0.5	13.105	13.1805	13.48578	13.2405	13.328	31.875	100	b

Note: The last column (cld for "compact letter display") is only displayed if the `multcomp` package is installed.

It provides ranks for the different times, allowing for ties.

Measuring the time used by an expression (II)

Another command:

```
system.time(expression)
```

which returns three numbers:

- user*: the time used to execute the expression itself
- system*: the time used by the system while executing the expression (e.g. time spent reading files)
- elapsed*: the total time spent
(the one we are usually interested in)

Comparing codes: version 1

```
for (i in 1:n) {  
  result <- mean( runif( m ) )  
  results <- c(results, result)  
}
```

Comparing code: version 2

```
for (i in 1:n) {  
  result <- mean( runif( m ) )  
  results[i] <- result  
}
```

Comparing the two versions

})

Comparing the two versions

```
} )
```

	user	system	elapsed
<code>results <- c(results, result)</code>	21.433	1.264	22.778
<code>results[i] <- result</code>	1.780	0.000	1.782

Can we improve the code further ?

```
for (i in 1:n) {  
  result <- mean( runif( m ) )  
  results[i] <- result  
}
```

One possible improvement: removing a temporary variable

```
for (i in 1:n) {  
    results[i] <- mean( runif( m ) )  
}
```


Comparing the three versions

	user	system	elapsed
<code>results <- c(results, result)</code>	21.433	1.264	22.778
<code>results[i] <- result</code>	1.780	0.000	1.782
<code>results[i] <- mean(runif(m))</code>	1.832	0.000	1.836

Is there any practical difference between these two loops ?

```
set.seed(1)
n <- 5000; m <- 5000
```

```
}  
}
```

```
# Loop 2  
for (i in 1:ncol(a)) {  
  for (j in 1:nrow(a)) {  
    b <- a[j,i]  
  }  
}
```

Is there any practical difference between these two loops ?

```
set.seed(1)
n <- 5000; m <- 5000
```

```
}
}
```

10 17 10 10

```
# Loop 2
for (i in 1:ncol(a)) {
  for (j in 1:nrow(a)) {
    b <- a[j,i]
  }
}
```

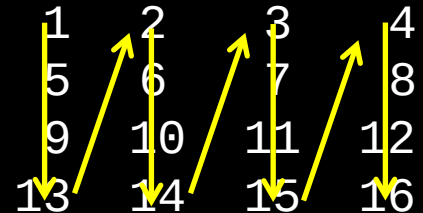
Is there any practical difference between these two loops ?

```
set.seed(1)
n <- 5000; m <- 5000
```

```
}
}
```

```
# Loop 2
for (i in 1:ncol(a)) {
  for (j in 1:nrow(a)) {
    b <- a[j,i]
  }
}
```

13 14 15 16



Is there any practical difference between these two loops ?

```
set.seed(1)
n <- 5000; m <- 5000
```

```
}
}
```

```
system.time(
  for (i in 1:ncol(a)) {
    for (j in 1:nrow(a)) {
      b <- a[j,i]
    }
  }
)
```

“The plural of anecdotes is not data”

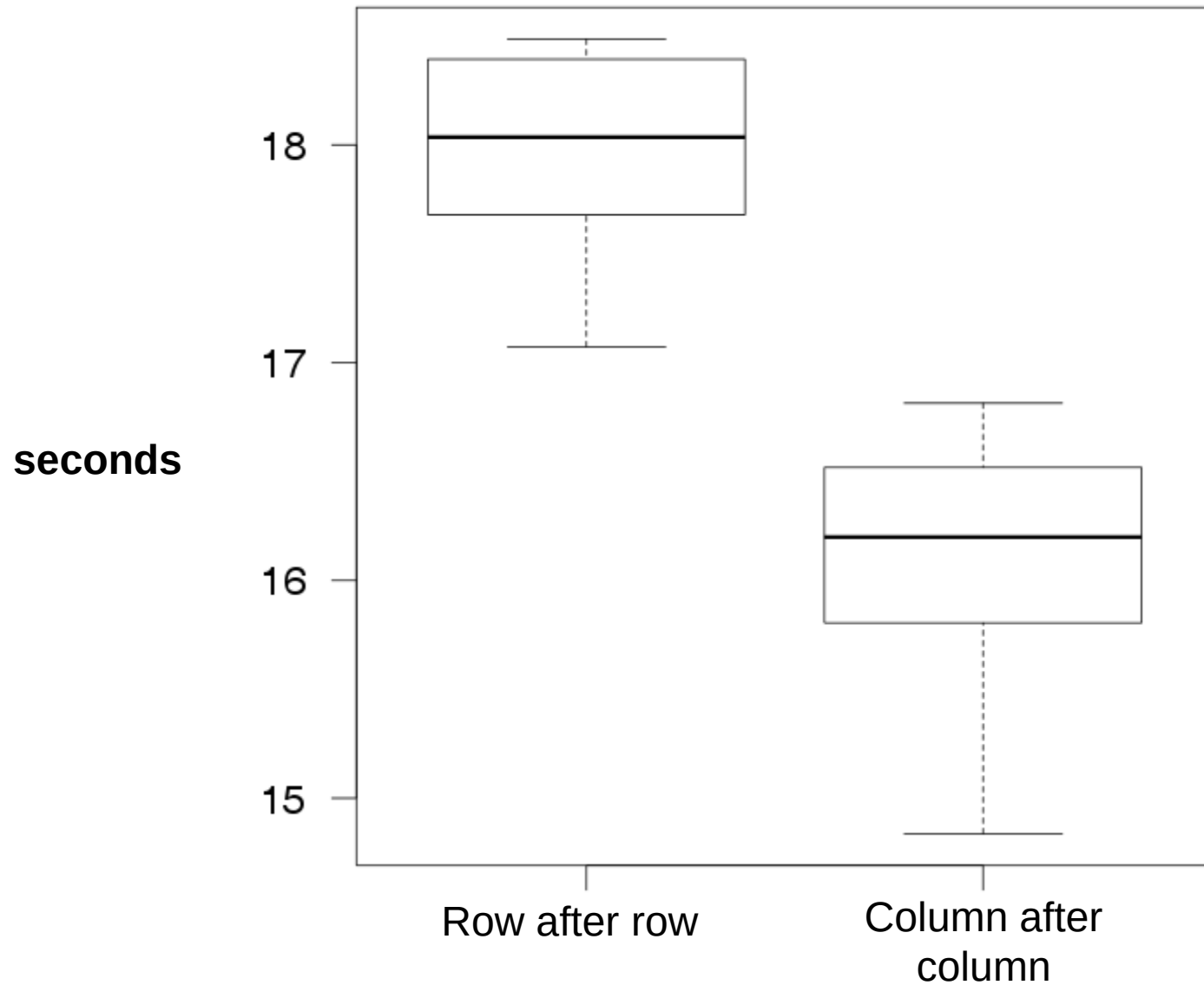
```
system.time(  
  for (i in 1:nrow(a)) {
```

```
    16.309    0.000    16.420
```

```
system.time(  
  for (i in 1:ncol(a)) {  
    for (j in 1:nrow(a)) {  
      b <- a[j,i]  
    }  
  }  
)
```

```
user  system elapsed  
16.281    0.000    16.308
```

After repeating the test several times under different circumstances



Profiling is a tool that allows the user to know how much time was spent on each part of his code.

It works by gathering information about what the code is doing at regular intervals (by default: every 20 ms, or 50 times per second) and saves it into the file.

Analyzing this file allows the user to find out which parts were slowest and may have to be rethought.


```
pval <- ttest$p.value  
pvalues <- c(pvalues, pval)  
}  
Rprof(NULL)
```

Displaying the results of the profiling

```
summaryRprof()
```

"mean"	0.18	6.52	0.24	8.70
"var"	0.16	5.80	0.44	15.94
"stopifnot"	0.12	4.35	0.18	6.52
"pmatch"	0.12	4.35	0.12	4.35
"t.test"	0.10	3.62	2.60	94.20
"paste"	0.08	2.90	0.92	33.33
"mode"	0.08	2.90	0.54	19.57
"c"	0.08	2.90	0.08	2.90
"pt"	0.08	2.90	0.08	2.90
"match.arg"	0.06	2.17	0.38	13.77
...				

Displaying the results of the profiling

```
summaryRprof()
```

"match"	0.20	7.25	0.64	23.19
"mode"	0.08	2.90	0.54	19.57
"var"	0.16	5.80	0.44	15.94
"match.arg"	0.06	2.17	0.38	13.77
".deparseOpts"	0.24	8.70	0.30	10.87
"mean"	0.18	6.52	0.24	8.70
"stopifnot"	0.12	4.35	0.18	6.52
"pmatch"	0.12	4.35	0.12	4.35
"c"	0.08	2.90	0.08	2.90
"pt"	0.08	2.90	0.08	2.90
...				

Displaying the results of the profiling

```
summaryRprof()
```

"mean"	0.18	6.52	0.24	8.70
"var"	0.16	5.80	0.44	15.94
"stopifnot"	0.12	4.35	0.18	6.52
"pmatch"	0.12	4.35	0.12	4.35
"t.test"	0.10	3.62	2.60	94.20
"paste"	0.08	2.90	0.92	33.33
"mode"	0.08	2.90	0.54	19.57
"c"	0.08	2.90	0.08	2.90
"pt"	0.08	2.90	0.08	2.90
"match.arg"	0.06	2.17	0.38	13.77
...				

Displaying the results of the profiling

```
summaryRprof()
```

"mean"	0.18	6.52	0.24	8.70
"var"	0.16	5.80	0.44	15.94
"stopifnot"	0.12	4.35	0.18	6.52
"pmatch"	0.12	4.35	0.12	4.35
"t.test"	0.10	3.62	2.60	94.20
"paste"	0.08	2.90	0.92	33.33
"mode"	0.08	2.90	0.54	19.57
"c"	0.08	2.90	0.08	2.90
"pt"	0.08	2.90	0.08	2.90
"match.arg"	0.06	2.17	0.38	13.77
...				

- Markus Schmidberger, Martin Morgan, Dirk Eddelbuettel, Hao Yu, Luke Tierney, Ulrich Mansmann. “State of the Art in Parallel Computing with R”. Journal of Statistical Software 2009: JSS
- The CRAN Task View: High-Performance and Parallel Computing with R

Data manipulation/aggregation

Mapping a function to a matrix : `apply()`

```
> m
      [,1] [,2] [,3] [,4] [,5] [,6]
[1,]     1     6    11    16    21    26
[2,]     2     7    12    17    22    27
[3,]     3     8    13    18    23    28
[4,]     4     9    14    19    24    29
[5,]     5    10    15    20    25    30
> apply(m, MAR=1, FUN=sum, na.rm=TRUE)
[1]  81  87  93  99 105
> rowSums(m)
[1]  81  87  93  99 105
```

`apply()` is generally faster than looping over all rows/columns. More specialized functions (e.g. `rowSums`) may be faster still.

Mapping a function to a matrix : apply()

```
> m
      [,1] [,2] [,3] [,4] [,5] [,6]
[1,]     1     6    11    16    21    26
[2,]     2     7    12    17    22    27
[3,]     3     8    13    18    23    28
[4,]     4     9    14    19    24    29
[5,]     5    10    15    20    25    30

> apply(m, MAR=2, FUN=function(x) { c(mean(x), median(x)) } )
      [,1] [,2] [,3] [,4] [,5] [,6]
[1,]     3     8    13    18    23    28
[2,]     3     8    13    18    23    28
```

If the function returns more than one value for each row or column, `apply` will automatically create a matrix instead of a vector.

Mapping a function to a list : lapply()

```
> n <- as.list(as.data.frame(m)); n
$V1
[1] 1 2 3 4 5

$V2
[1] 6 7 8 9 10
...
> lapply(n, FUN=sum)
$V1
[1] 15

$V2
[1] 40
...

> sapply(n, FUN=sum)
 V1  V2  V3  V4  V5  V6
15  40  65  90 115 140
```

`lapply( )` and `sapply( )` both map a function to each element of a list; the first one returns a list, the other returns a vector or an array

How can we map a function to different groups ?

4	M	175
5	M	158
6	M	179

Mapping a function to groups

```
> head(data)
  sex height
1  M     183
2  M     183
3  M     182
4  M     175
5  M     158
6  M     179
> tapply(data$height, data$sex, FUN=mean)
      F      M
166.1739 178.2500
```

Returns a vector or a list, depending on the output of the function (scalar or more complex object)

Mapping a function to groups given by several factors

```
4      M      175 nonsmoker
5      M      158 nonsmoker
6      M      179  smoker
```

```
> tapply(data$height, list(data$sex, data$smoking), FUN=mean)
  nonsmoker smoker
F  166.3500    165
M  178.8421    176
```

The aggregate() function

aggregate(x, by, FUN)

Data frame List Function

`aggregate()` works in a similar way to `tapply()`, but

- It works on whole data frames (multiple columns)
- It can only produce scalar summaries

The aggregate() function

```
> data(iris)
> head(iris, 3)
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5         1.4         0.2   setosa
2          4.9         3.0         1.4         0.2   setosa
3          4.7         3.2         1.3         0.2   setosa
> aggregate(iris[, 1:4], iris[5], FUN=mean)
  Species Sepal.Length Sepal.Width Petal.Length Petal.Width
1   setosa      5.006      3.428      1.462      0.246
2 versicolor      5.936      2.770      4.260      1.326
3  virginica      6.588      2.974      5.552      2.026
```

Note that the by argument is `iris[5]` (a list, or a data frame column) and not `iris[,5]` (a vector or factor)

Splitting data according to groups information

```
> split( data[, "height"], data[, "sex"] )  
$F  
 [1] 172 165 165 156 172 168 166 176 159 164 150 163 169 160  
[15] 165 170 173 165 159 175 170 168 172  
  
$M  
 [1] 183 183 182 175 158 179 185 177 186 178 183 178 183 177  
[15] 180 174 184 168 169 181 170 180 184 181
```

Splits a vector (or rows of a data.frame) into separate elements of a list, ready for further processing.

Assign and retrieve variables «on the fly»

```
< assign(paste0( treatment , patient10),  
        sample( c("control", "treatment"), 1) )  
> treatment10  
[1] "treatment"
```

Use eval to create R commands «on the fly»

```
for (i in 1:12) {  
  eval(parse(text=paste0("temp.",i," <- c(",i,",",",i,")")))  
}
```

```
> ls()  
[1] "i"          "temp.1"    "temp.10"   "temp.11"   "temp.12"  
[6] "temp.2"    "temp.3"    "temp.4"    "temp.5"    "temp.6"  
[11] "temp.7"    "temp.8"    "temp.9"  
> temp.1  
[1] 1 1  
> temp.10  
[1] 10 10
```

A concrete example: code executed depends on data

Data indicates which R code should be executed.

```
> x <- runif(10); y <- rnorm(10)
> data
  model formula
1    lm      y~x
2   glm      y~x

for (i in 1:nrow(data)) {
  eval( parse( text=paste0( "model", i, " <- ", data$model[i],
                           "(", data$formula[i], ")") ) )
}

> ls()
[1] "data"    "i"       "model1"  "model2"  "x"       "y"
> model1

Call:
lm(formula = y ~ x)
...
```

R best practices

What we are not going to discuss: style rules

- How to format your code
- How to indent your code
- How to name your identifiers
- How to use comments
- ...

Suggestion of R styleguides:

- Google
<https://google.github.io/styleguide/Rguide.xml>
- Hadley Wickham
<http://adv-r.had.co.nz/Style.html>

| >

vs

%>%

A typical R command can contain many sets of parenthesis:

```
> log(round(exp(2), 1))  
[1] 2.00148
```

To understand the command, need to read it from
right to left / inside to outside

```
> log(round(exp(2), 1))
```


R as a functional language

`exp(2) = 7.389`

```
> log(round(exp(2), 1))
```

R as a functional language

`exp(2) = 7.389`

```
> log(round(7.389 , 1))
```

R as a functional language

`round(7.389, 1) = 7.4`

```
> log(round(7.389 , 1))
```

R as a functional language

`round(7.389, 1) = 7.4`

```
> log(7.4)
```

R as a functional language

$$\log(7.4) = 2.00148$$

```
> log(7.4)
```

R as a functional language

$$\log(7.4) = 2.00148$$

```
> log(7.4)
[1] 2.00148
```



Since 2012, R packages have provided a pipe operator (most commonly `%>%`) to allow us to rewrite this command as a chain of operations :

```
> library(magrittr)
> 2 %>%
+   exp() %>%
+   round(1) %>%
+   log()
[1] 2.00148
```

The result of each command is the first argument for the next command

```
> library(magrittr)
> 2 %>%
+   exp() %>%
+   round(1) %>%
+   log()
[1] 2.00148
```

Equivalent to:

```
exp(2) = 7.389...
round(7.389, 1) = 7.4
log(7.4)
```


Storing the results in a variable

The result can be saved as usual in R:

```
> library(magrittr)
> result <- 2 %>%
+   exp() %>%
+   round(1) %>%
+   log()
> result
[1] 2.00148
```

Equivalent to:

```
exp(2) = 7.389...
round(7.389, 1) = 7.4
log(7.4)
```

Storing the results in a variable

... or using the lesser known reverse arrow:

```
> library(magrittr)
> 2 %>%
+   exp() %>%
+   round(1) %>%
+   log() -> result
> result
[1] 2.00148
```

Equivalent to:

```
exp(2) = 7.389...
round(7.389, 1) = 7.4
log(7.4)
```

Since R 4.1 (May 2021), R provides a native pipe operator: `|>`

```
> 2 |>  
+   exp() |>  
+   round(1) |>  
+   log()  
[1] 2.00148
```



Which one should you use ?

|>

- This is part of base R, so available to everyone
- The native pipe is simpler than %>%
- However, it can not be used with R < 4.1

%>%

- The Magrittr pipe allows more complicated structures

Suggestion:

- Use the base R pipe (|>) by default
- Switch to %>% if you really need an advanced feature

See <https://r4ds.hadley.nz/workflow-pipes.html>



```
> head(clinicaldata, n=3)
  phenotype age sex weight
1  4.373546 NA  F     77
2  5.183643 46  M     89
3  4.164371 52  M     76

> subset(clinicaldata, sex=="F" & age <40, select=-weight)
  phenotype age sex
8   5.738325 39  F
16  4.955066 24  F
17  4.983810 32  F
20  5.593901 36  F
```

The subset commands allows the selection of rows (or elements of vectors) based on logical expressions, and selection of columns based on names.

It removes NA values from columns where a selection is done.

The subset function is useful when working in an interactive session, but its use is not recommended in scripts, according to the help page:

Warning:

This is a convenience function intended for use interactively. For programming it is better to use the standard subsetting functions like '[', and in particular the non-standard evaluation of argument 'subset' can have unanticipated consequences.

```
# Generate a dataset
set.seed(1)
x <- runif(100)
y <- 2*x + rnorm(length(x))/10
data <- data.frame(x, y)

# Fit a linear model
model <- lm( data$y ~ data$x )

# Generate a second dataset
x <- runif(100)
y <- 2*x + rnorm(length(x))/10
newdata <- data.frame(x, y)

# Use the linear model to perform a prediction on the newdata
predict(model, newdata)
```

This code does not return any error message, but it does not work. Why ?

```
> model <- lm( data$y ~ data$x )
> summary(model)
```

```
Call:
lm(formula = data$y ~ data$x)
```

```
Residuals:
```

	Min	1Q	Median	3Q	Max
	-0.184978	-0.056222	-0.008707	0.052427	0.251661

```
Coefficients:
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-0.01793	0.02058	-0.871	0.386
data\$x	2.03123	0.03535	57.467	<2e-16 ***

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 0.09411 on 98 degrees of freedom
```

```
Multiple R-squared:  0.9712,    Adjusted R-squared:  0.9709
```

```
F-statistic: 3303 on 1 and 98 DF,  p-value: < 2.2e-16
```



```
> model <- lm( data$y ~ data$x )
> summary(model)
```

```
Call:
lm(formula = data$y ~ data$x)
```

```
Residuals:
```

	Min	1Q	Median	3Q	Max
	-0.184978	-0.056222	-0.008707	0.052427	0.251661

```
Coefficients:
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-0.01793	0.02058	-0.871	0.386
data\$x	2.03123	0.03535	57.467	<2e-16 ***

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 0.09411 on 98 degrees of freedom
Multiple R-squared:  0.9712,    Adjusted R-squared:  0.9709
F-statistic: 3303 on 1 and 98 DF,  p-value: < 2.2e-16
```

```
> model <- lm( y ~ x, data=data )
> summary(model)
```

```
Call:
lm(formula = y ~ x, data = data)
```

```
Residuals:
```

	Min	1Q	Median	3Q	Max
	-0.184978	-0.056222	-0.008707	0.052427	0.251661

```
Coefficients:
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-0.01793	0.02058	-0.871	0.386
x	2.03123	0.03535	57.467	<2e-16 ***

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 0.09411 on 98 degrees of freedom
```

```
Multiple R-squared:  0.9712,    Adjusted R-squared:  0.9709
```

```
F-statistic: 3303 on 1 and 98 DF,  p-value: < 2.2e-16
```

The correct way to do it

```
# Generate a dataset
```

```
# Generate a second dataset
```

```
x <- runif(100)
```

```
y <- 2*x + rnorm(length(x))/10
```

```
newdata <- data.frame(x, y)
```

```
# Use the linear model to perform a prediction on the newdata
```

```
predict(model, newdata)
```

```
> annotations <- read.table("annotations-from-provider.txt")
  identifier entrezid gene
1         31   73398   H
2         41   55359   P
3         89   97377   H
4         63   37348   Y
5         17    4465   T
6         55   55583   Z
7         55   17866   K
...
# We do not need the gene code
> annotations <- annotations[,1:2]
```

If available, always use data frame names instead of column numbers:

```
> annotations <- annotations[, c("identifier", "entrezid") ]
```

Loading file affy-annot.txt into R

	ProbeSet ID	ID	Target Description
1	1007_s_at	U48705	discoidin domain receptor tyrosine kinase 1
2	1053_at	M87338	replication factor C (activator 1) 2, 40kDa
3	117_at	X51757	heat shock 70kDa protein 6 (HSP70B')
4	121_at	X69699	paired box 8
5	1255_g_at	L36861	guanylate cyclase activator 1A (retina)
6	1294_at	L13852	ubiquitin-like modifier activating enzyme 7
7	1487_at	L38487	Human ER-related protein (hERRa1) mRNA, 3' end
8	1316_at	X55005	thyroid hormone receptor, alpha
9	1320_at	X79510	protein tyrosine phosphatase, non-receptor type 21
10	1405_i_at	M21121	chemokine (C-C motif) ligand 5
11	1431_at	J02843	cytochrome P450, family 2, subfamily E, polypeptide 1
12	1438_at	X75208	EPH receptor B3

```
> data <- read.table("affy-annot.txt", sep="\t")
> dim(data)
[1] 8 3
```

Where are the 4 missing rows?

Loading file affy-annot.txt into R

	ProbeSet ID	ID	Target Description
1	1007_s_at	U48705	discoidin domain receptor tyrosine kinase 1
2	1053_at	M87338	replication factor C (activator 1) 2, 40kDa
3	117_at	X51757	heat shock 70kDa protein 6 (HSP70B')
4	121_at	X69699	paired box 8
5	1255_g_at	L36861	guanylate cyclase activator 1A (retina)
6	1294_at	L13852	ubiquitin-like modifier activating enzyme 7
7	1487_at	L38487	Human ER-related protein (hERRa1) mRNA, 3' end
8	1316_at	X55005	thyroid hormone receptor, alpha
9	1320_at	X79510	protein tyrosine phosphatase, non-receptor type 21
10	1405_i_at	M21121	chemokine (C-C motif) ligand 5
11	1431_at	J02843	cytochrome P450, family 2, subfamily E, polypeptide 1
12	1438_at	X75208	EPH receptor B3

```
> data <- read.table("affy-annot.txt", sep="\t")
> dim(data)
[1] 8 3
```

Where are the 4 missing rows?

Loading file affy-annot.txt into R

	ProbeSet ID	ID	Target Description
1	1007_s_at	U48705	discoidin domain receptor tyrosine kinase 1
2	1053_at	M87338	replication factor C (activator 1) 2, 40kDa
3	117_at	X51757	heat shock 70kDa protein 6 (HSP70B'...' end
8	1316_at	X55005	thyroid hormone receptor, alpha
9	1320_at	X79510	protein tyrosine phosphatase, non-receptor type 21
10	1405_i_at	M21121	chemokine (C-C motif) ligand 5
11	1431_at	J02843	cytochrome P450, family 2, subfamily E, polypeptide 1
12	1438_at	X75208	EPH receptor B3

```
> data <- read.table("affy-annot.txt", sep="\t")  
> dim(data)  
[1] 8 3
```

The 4 missing lines are all in the 3rd row.

- Assume as little as possible about your data
- In particular, always specify the «quote» argument when reading a file (especially if you do not use quotes !):

```
> data <- read.table("affy-annot.txt", sep="\t", quote="" )  
> dim(data)  
[1] 12  3
```

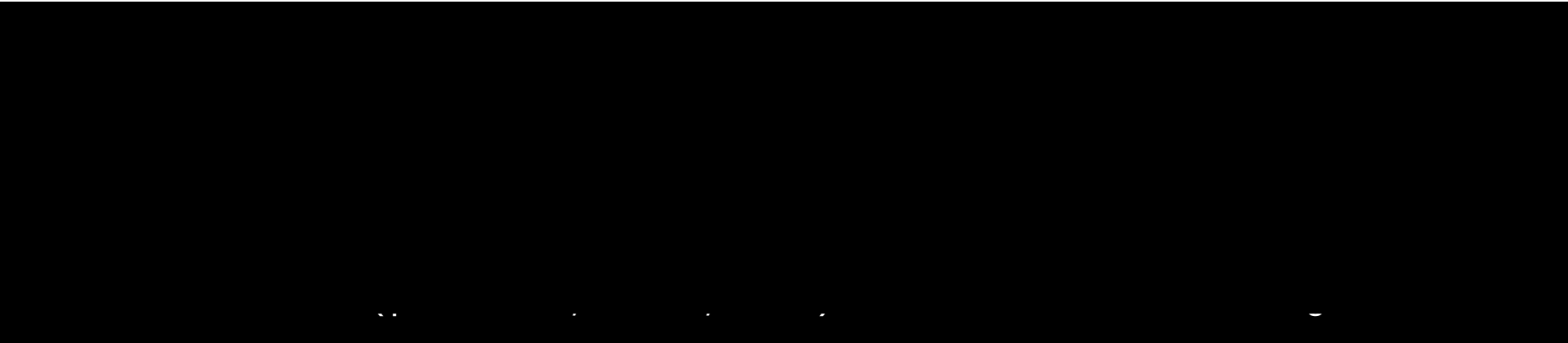

Reproducible research in R

6.1.3 *Graphical presentation*

Of course, there are many ways to present grouped data. Here we create a somewhat elaborate plot where the raw data are plotted as a stripchart and overlaid with an indication of means and SEMs (Figure 6.1):

```
> xbar <- tapply(folate, ventilation, mean)
> s <- tapply(folate, ventilation, sd)
> n <- tapply(folate, ventilation, length)
> sem <- s/sqrt(n)
> stripchart(folate~ventilation, "jitter", jit=0.05, pch=16, vert=T)
> arrows(1:3, xbar+sem, 1:3, xbar-sem, angle=90, code=3, length=.1)
> lines(1:3, xbar, pch=4, type="b", cex=2)
```

The code does not work in R 2.7+



CHANGES IN R VERSION 2.7.0

GRAPHICS CHANGES

- o `stripchart()` is now a generic function, with default and formula methods defined. Additional graphics parameters may be included in the call. Formula handling is now similar to `boxplot()`.

1-D Scatter Plots

Description:

'stripchart' produces one dimensional scatter plots (or dot plots) of the given data. These plots are a good alternative to 'boxplot's when sample sizes are small.

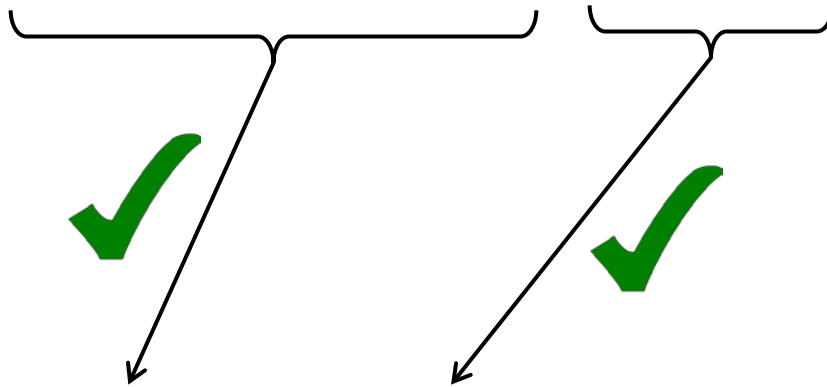
Usage:

```
stripchart(x, method = "overplot", jitter = 0.1, offset = 1/3,  
           vertical = FALSE, group.names, add = FALSE,  
           at = NULL, xlim = NULL, ylim = NULL,  
           ylab=NULL, xlab=NULL, dlab="",  
           log = "", pch = 0, col = par("fg"), cex = par("cex"))
```

Arguments:

x: the data from which the plots are to be produced. The data can be specified as a single numeric vector, or as list of numeric vectors, each corresponding to a component plot. Alternatively a symbolic specification of the form 'x ~ g' can be given, indicating the observations in the vector 'x' are to be grouped according to the levels of the factor 'g'.

```
stripchart(folate~ventilation, "jitter", jit=0.05, pch=16, vert=T)
```



```
stripchart(x, method = "overplot", jitter = 0.1, offset = 1/3,  
vertical = FALSE, group.names, add = FALSE,  
at = NULL, xlim = NULL, ylim = NULL,  
ylab=NULL, xlab=NULL, dlab="",  
log = "", pch = 0, col = par("fg"), cex = par("cex"))
```

Arguments:

`x`: the data from which the plots are to be produced. The data can be specified as a single numeric vector, or as list of numeric vectors, each corresponding to a component plot. Alternatively a **symbolic specification of the form 'x ~ g' can be given**, indicating the observations in the vector 'x' are to be grouped according to the levels of the factor 'g'.

1-D Scatter Plots

Description:

'stripchart' produces one dimensional scatter plots (or dot plots) of the given data. These plots are a good alternative to 'boxplot's when sample sizes are small.

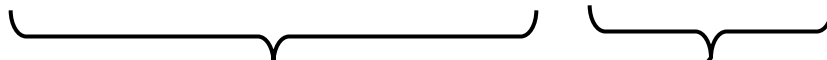
Usage:

```
stripchart(x, ...)
```

```
## S3 method for class 'formula'  
stripchart(x, data = NULL, dlab = NULL, ...,  
           subset, na.action = NULL)
```

```
## Default S3 method:  
stripchart(x, method = "overplot", jitter = 0.1, offset = 1/3,  
           vertical = FALSE, group.names, add = FALSE,  
           at = NULL, xlim = NULL, ylim = NULL,  
           ylab=NULL, xlab=NULL, dlab="", glab="",  
           log = "", pch = 0, col = par("fg"), cex = par("cex"),  
           axes = TRUE, frame.plot = axes, ...)
```

```
stripchart(folate~ventilation, "jitter", jit=0.05, pch=16, vert=T)
```



```
stripchart(x, data = NULL, dlab = NULL, ...,  
          subset, na.action = NULL)
```

Default S3 method:

```
stripchart(x, method = "overplot", jitter = 0.1, offset = 1/3,  
          vertical = FALSE, group.names, add = FALSE,  
          at = NULL, xlim = NULL, ylim = NULL,  
          ylab=NULL, xlab=NULL, dlab="", glab="",  
          log = "", pch = 0, col = par("fg"), cex = par("cex"),  
          axes = TRUE, frame.plot = axes, ...)
```


Corrected code in the second edition of the book

```
> library(ISwR)
> data(red.cell.folate)
> attach(red.cell.folate)
> stripchart(folate~ventilation, "jitter", jit=0.05, pch=16,
+ vert=T)
Error in eval(predvars, data, env) : invalid 'envir' argument

> stripchart(folate~ventilation, method="jitter",
+ jitter=0.05, pch=16, vert=T)
```

Also worth noting: the short parameter «`jit`» has been replaced by the full name «`jitter`»

- Rules for passing argument: only the first one by position, all the other by name, and never in an abbreviated way.

CHANGES IN R VERSION 2.4.0

USER-VISIBLE CHANGES

- o The functions `read.csv()`, `read.csv2()`, `read.delim()`, `read.delim2()` now default their 'comment.char' argument to `"`. (These functions are designed to read files produced by other software, which might use the `#` character inside fields, but are unlikely to use it for comments.)

CHANGES IN R VERSION 2.4.0

USER-VISIBLE CHANGES

- o The functions `read.csv()`, `read.csv2()`, `read.delim()`, `read.delim2()` now default their 'comment.char' argument to `"`. (These functions are designed to read files produced by other software, which might use the `#` character inside fields, but are unlikely to use it for comments.)

How to run R scripts from the command line

Calling R from the command line

```
C:\Program Files\R\R-3.0.1\bin\Rscript.exe script.R
```

Previously

```
C:\Program Files\R\R-3.0.1\bin\R.exe CMD BATCH script.R
```

Example script

```
print(mean(a))  
cat("Done\n")
```

```
schutz@laptop:~/batch$ R CMD BATCH script.R
schutz@laptop:~/batch$ ls -a
total 28
script.R      script.Rout   .Rdata       asdf.png

schutz@laptop:~/batch$ Rscript script.R
```

- Rscript does not create a complete log (.Rout) of the output

R version 3.4.3 (2017-11-30) -- "Kite-Eating Tree"

```

null device
      1

```

```

>
> print(mean(a))
[1] 0.5114525
> cat("Done\n")

```

Done

```

>
> proc.time()
      user  system elapsed
  0.183    0.035    0.208

```

```
R CMD BATCH --vanilla --slave script.R outputfile
```

```
Rscript script.R # equivalent to
```

```
Rscript --no-restore --slave script.R
```

--vanilla means:

- no-save

- no-restore

- no-site-file

- no-init-file

- no-environ

Parameters ?

```
Rscript script 1 2
```

```
R CMD BATCH --vanilla '--args a=1 b=c(2,5,6)' script.R
```

From the script

```
args <- commandArgs(TRUE)
a <- as.double(args[1])
b <- as.double(args[2])
```

- See also `getOpts`

```
# on Linux/Mac
```

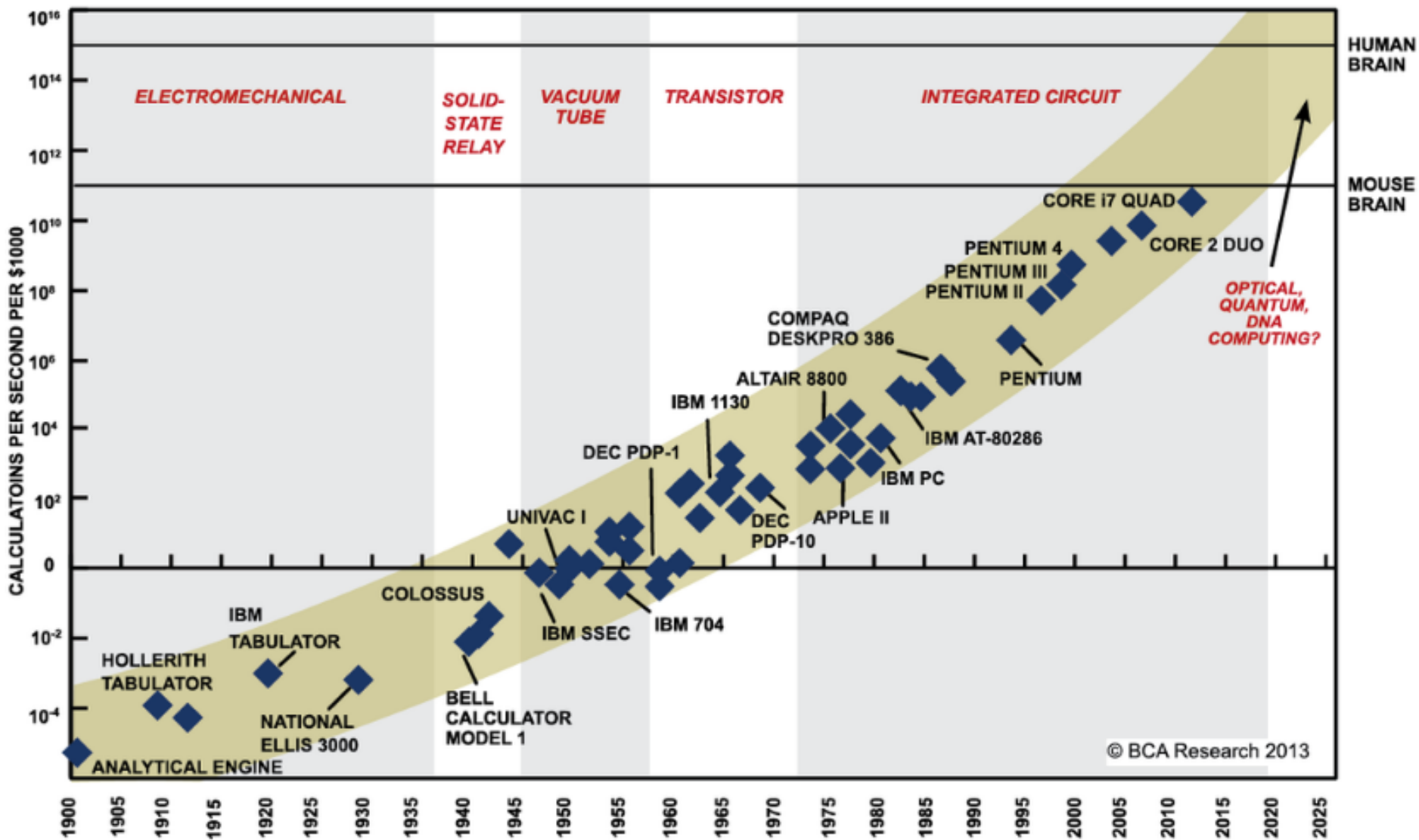
```
R CMD BATCH [options] my_script.R [outfile]
```

```
# on Microsoft Windows (adjust the path to R.exe as needed)
```

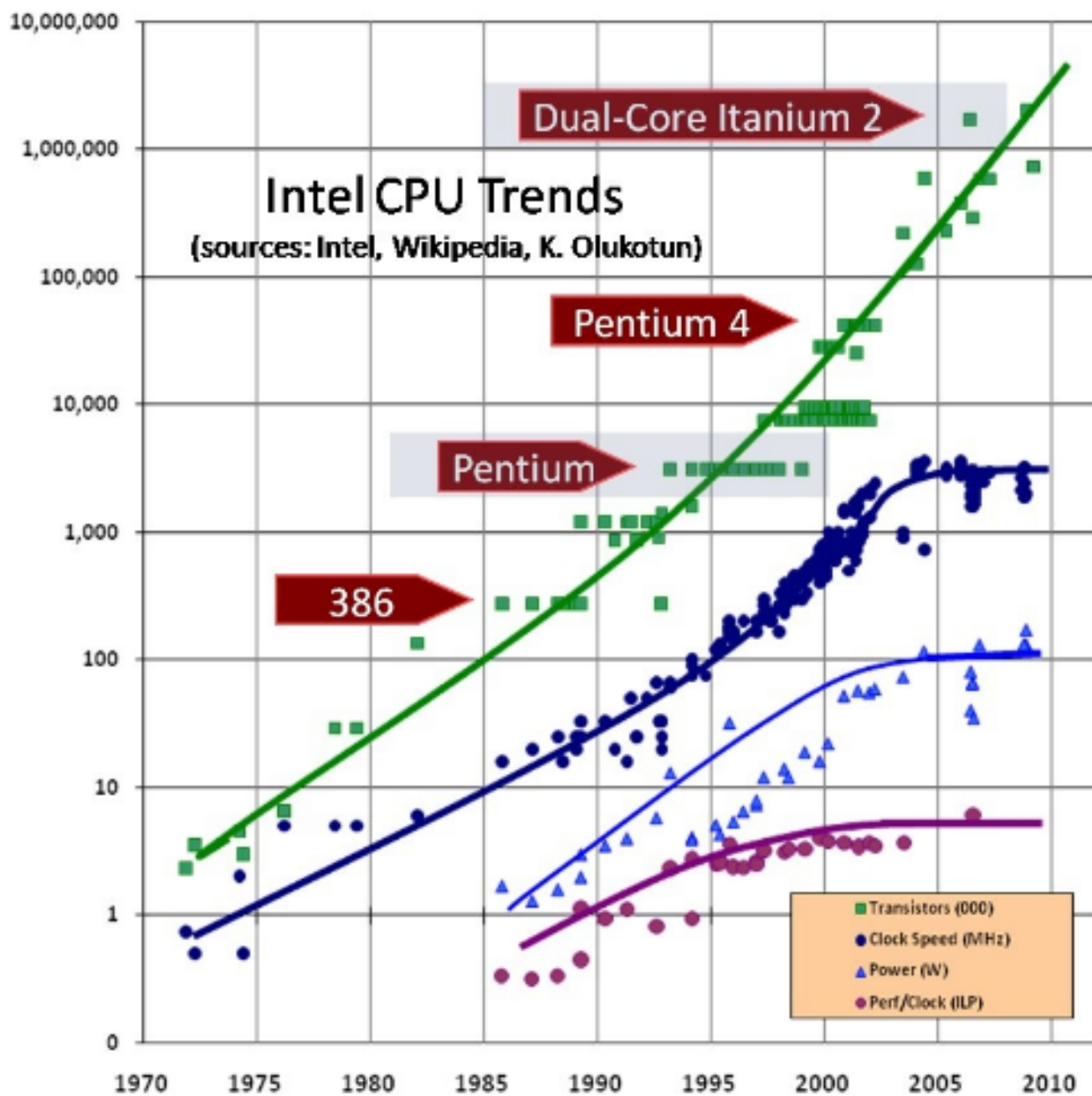
```
"C:\Program Files\R\R-2.13.1\bin\R.exe" CMD BATCH  
  --vanilla --slave "c:\my projects\my_script.R"
```

An introduction to parallelization in R

**(with thanks to
Geoffrey Fucile)**



SOURCE: RAY KURZWEIL, "THE SINGULARITY IS NEAR: WHEN HUMANS TRANSCEND BIOLOGY", P.67, THE VIKING PRESS, 2006. DATAPOINTS BETWEEN 2000 AND 2012 REPRESENT BCA ESTIMATES.



**Increase in computing
performance nowadays comes
from parallelization rather than
increase of processor
performances**

When to use parallelization ?

When to use parallelization ?

- Independent data/calculations,
“embarrassingly parallel”
- Bootstrapping and cross validation
- If the gain in time is more than spent on code implementation, data transfer etc

**It is not a silver bullet
and it is (generally) not automatic,
(never) magical
(often) not easy.**

Program reduced to a repetitive structure

```
for (i in 1:n) {  
    do_calculation()  
}
```

When to parallelize ?

- If the calculations are **completely independent**
- If I can remove/reduce the non independence

Two ways of parallelizing code



Open Multi-Processing

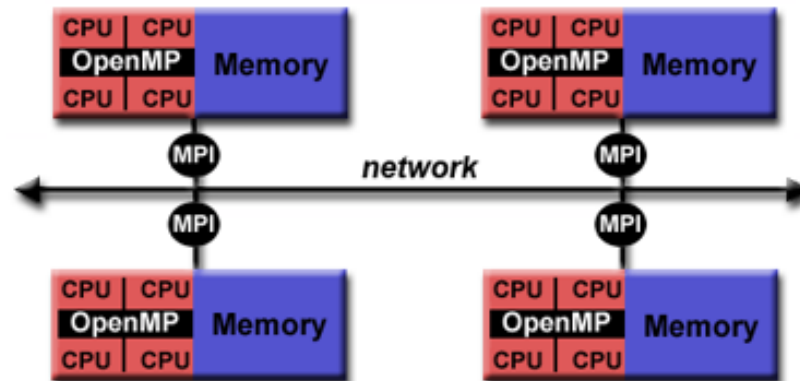


Message Passing Interface

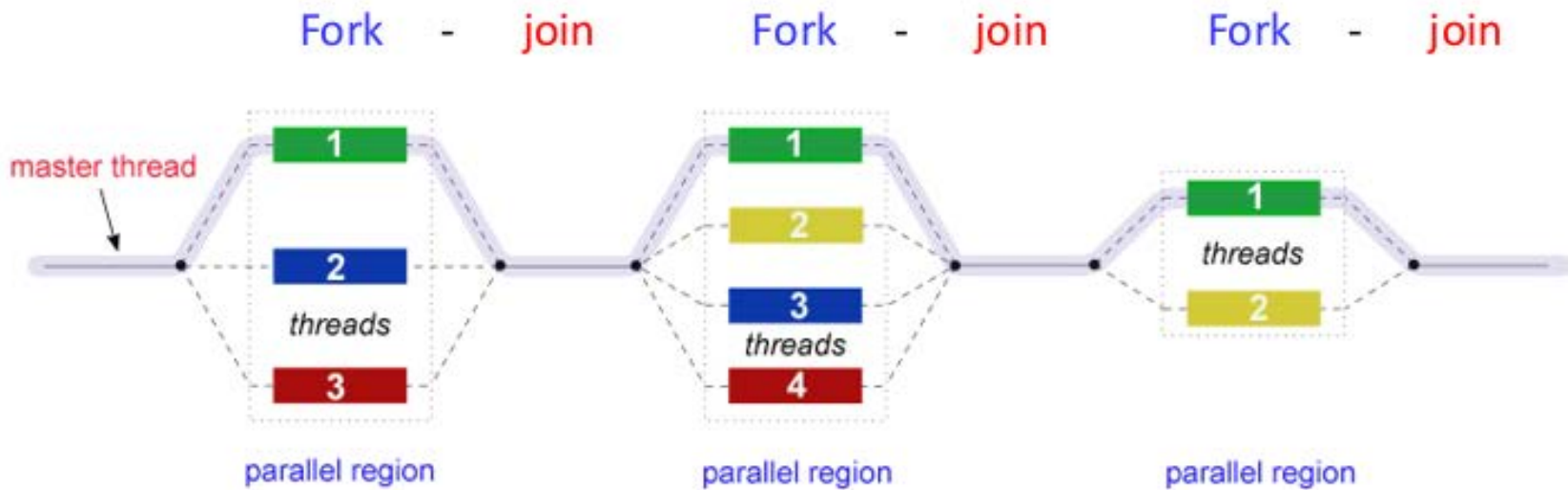
Both are APIs for C/C++ and Fortran

Works in **shared** memory systems
Can use as many cores as there are in the **compute node**
Uses **threads** to split the work
Threads have both **private and shared** variables
Uses **preprocessor directives** in commented lines

Works in **shared and distributed** memory systems
Can use as many cores as there are in the **cluster**
Uses **processes** to split the work
Processes have only **private** variables
Loads a library and uses **specific commands**



Typical parallelization



- R can implement both OpenMP and MPI types of parallelization
- More commonly, tasks are parallelized in R using FORK or PSOCK protocols
- The only “universal” implementation is PSOCK, as it works on both UNIX-type and Windows environments

- Parallelization methods in R are available through the parallel and doParallel packages
 - built on several predecessor packages, including snow and multicore among others
 - snow is only necessary for Windows, which uses PSOCK

- The foreach package allows parallelized iteration of a workflow, and requires a parallel backend to be registered by the doMC package, doParallel, doMPI, etc
- doMC is designed for parallel/multicore processing on a “single workstation”
 - This is ideal for a compute cluster, where the compute node environment can be viewed as a single workstation by doMC
 - The parallel backend is created using a system call to fork()

How many cores can I use on my (virtual) machine ?

```
> library(parallel)
> detectCores()
[1] 4
```



Using the parallel package: lapply

```
## user system elapsed  
## 0.047 0.005 0.096
```

```
stopCluster(cl)
```

```
# VERY IMPORTANT!
```

```
class(result)
```

```
## [1] "list"
```



Using the parallel package: sapply

```
## user system elapsed  
## 0.058 0.008 0.134
```

```
stopCluster(cl) # VERY IMPORTANT!
```

```
class(result)  
## [1] "numeric"
```

A serial foreach loop

```
library(foreach)
```

```
(system.time(  
results <- foreach(1:5) %do% {  
  r1 <- rnorm(1e7); r1.m <- mean(r1); return(r1.m);  
})  
)  
##   user  system elapsed  
## 4.043   0.107   4.165
```

```
library(doParallel)
library(foreach)
cl <- makeCluster(2)
registerDoParallel(cl)

(system.time(
  results <- foreach(1:5) %dopar% {
    r1 <- rnorm(1e7); r1.m <- mean(r1); return(r1.m);
  })
)
##   user   system elapsed
## 0.253    0.069     3.424
stopCluster(cl)                                # VERY IMPORTANT!
```

Generating random numbers on a computer

Using a "real" random number generator (mostly for cryptography)



DigiSAFE DigiNoise
Random Number Generator



Generating random numbers for scientific simulations

In scientific simulations, we usually need sequences of **numbers that look random** (that is: looking at a series of number, we can not predict what the next one will be), but that remain **predictable and repeatable** when needed.

Otherwise, debugging is difficult, and it is impossible to verify the results obtained by others.

Using a pseudo-random number generator

A pseudo-random number generator (PRNG) fulfils this task; it usually includes two parts:

- A **seed**: an initial value
- A function that generates a new "random" number based on the previous ones (or on the seed)

Example: the linear congruential method

The series of random numbers is given by

$$X_{n+1} = (a X_n + c) \bmod m$$

where

a, c and m are (**well-chosen**) constants;

X_n is the previous random number (or the seed)

X_{n+1} is the next random number

A good example

$$X_{n+1} = (48271 X_n + c) \bmod (2^{31} - 1)$$

A bad example:

$$X_{n+1} = (65539 X_n) \bmod 2^{31}$$

Called RANDU, this generator was used in most of the computers for more than a decade; it actually fails most criteria for randomness !

Example

i	X_{i-1}	$(X_{i-1}) * 5 + 1$	$(X_{i-1}) * 5 + 1 \bmod 32$	X_i
0	1 (seed)	6	$6 = 32 * 0 + 6$	6
1	6	31	$31 = 32 * 0 + 31$	31
2	31	156	$156 = 32 * 4 + 28$	28
3	28	141	$141 = 32 * 4 + 13$	13
4	13	66	$66 = 32 * 2 + 2$	2
5	2	11	$11 = 32 * 0 + 11$	11
6	11	56	$56 = 32 * 1 + 24$	24
7	24	121	$121 = 32 * 3 + 25$	25
8	25	126	$126 = 32 * 3 + 30$	30

What happens in R ?

Random

package:base

R Documentation

Random Number Generation

Description:

'`.Random.seed`' is an integer vector, containing the random number generator (RNG) *state* for random number generation in R. It can be saved and restored, but should not be altered by the user.

'`RNGkind`' is a more friendly interface to query or set the kind of RNG in use.

'`RNGversion`' can be used to set the random generators as they were in an earlier R version (for reproducibility).

'`set.seed`' is the recommended way to specify seeds.

Details:

The currently available RNG kinds are given below. 'kind' is partially matched to this list. The default is `"Mersenne-Twister"`.

`"Wichmann-Hill"` The seed, `'.Random.seed[-1] == r[1:3]` is an integer vector of length 3, where each `'r[i]` is in `'1:(p[i] - 1)'`, where `'p'` is the length 3 vector of primes, `'p = (30269, 30307, 30323)'`. The Wichmann-Hill generator has a cycle length of $6.9536e12$ (`= 'prod(p-1)/4'`, see `_Applied Statistics_ (1984) *33*, 123` which corrects the original article).

`"Marsaglia-Multicarry"`: A `_multiply-with-carry_` RNG is used, as recommended by George Marsaglia in his post to the mailing list `'sci.stat.math'`. It has a period of more than 2^{60} and has passed all tests (according to Marsaglia). The seed is two integers (all values allowed).

'"Mersenne-Twister"' : From Matsumoto and Nishimura (1998). A twisted GFSR with period $2^{19937} - 1$ and equidistribution in 623 consecutive dimensions (over the whole period). The 'seed' is a 624-dimensional set of 32-bit integers plus a current position in that set.

'"Knuth-TAOCP-2002"' : A 32-bit integer GFSR using lagged Fibonacci sequences with subtraction. That is, the recurrence used is

$$X[j] = (X[j-100] - X[j-37]) \bmod 2^{30}$$

and the 'seed' is the set of the 100 last numbers (actually recorded as 101 numbers, the last being a cyclic shift of the buffer). The period is around 2^{129} .

Monte Carlo Simulations: Hidden Errors from “Good” Random Number Generators

Alan M. Ferrenberg and D. P. Landau

Center for Simulation Physics, The University of Georgia, Athens, Georgia 30602

Y. Joanna Wong

IBM Corporation, Supercomputing Systems, Kingston, New York 12401

(Received 29 July 1992)

The Wolff algorithm is now accepted as the best cluster-flipping Monte Carlo algorithm for beating “critical slowing down.” We show how this method can yield *incorrect* answers due to subtle correlations in “high quality” random number generators.

PACS numbers: 75.40.Mg, 05.70.Jk, 64.60.Fr

What about the seed ?

The seed should be random if we want random numbers (we need a bit of randomness to start the system, and it will then produce more randomness)

From the R help:

Initially, there is no seed; **a new one is created from the current time (and since R 2.14.0, the process ID)** when one is required. Hence different sessions will give different simulation results, by default. However, the seed might be restored from a previous session if a previously saved workspace is restored.

Record the seed used when generating random numbers

The `set.seed()` command allows one to choose a seed, so that the sequence of random numbers can be repeated.

Always record that seed, so that the results can be reproduced.

Suggestions:

`set.seed(1)`

for exercises

`set.seed(202604041)`

for real simulations

(easy, not duplicated, and reproducible)



SIGNIFICANT USER-VISIBLE CHANGES

- o The default method for generating from a discrete uniform distribution (used in `sample()`, for instance) has been changed. This addresses the fact, pointed out by Ottoboni and Stark, that the previous method made `sample()` noticeably non-uniform on large populations. See PR#17494 for a discussion. The previous method can be requested using `RNGkind()` or `RNGversion()` if necessary for reproduction of old results. [...]

The output of `RNGkind()` has been changed to also return the 'kind' used by `sample()`.

```
# To go back to the former number generator  
RNGkind(sample.kind = "Rounding")
```

```
# To use the new one  
RNGkind(sample.kind = "default")
```